[NOMBRE DE LA UNIVERSIDAD / INSTITUCIÓN]

[Facultad / Carrera]

[Asignatura: Redes de Computadoras]

Desarrollo de un simulador interactivo en Python para visualizar el encapsulamiento y desencapsulamiento de datos en las 7 capas del Modelo OSI

Manual Técnico · Manual de Usuario · Análisis de Resultados

Integrantes del grupo:

[Integrante 1 – Nombre completo – Código]

[Integrante 2 – Nombre completo – Código]

[Integrante 3 – Nombre completo – Código]

[Integrante 4 – Nombre completo – Código]

Docente: [Nombre del docente]

Repositorio GitHub: [https://github.com/usuario/simulador-osi]

Fecha: septiembre de 2026

Índice técnico

Índice técnico 2
1. Introducción 4
1.1. Objetivos 4
2. Arquitectura del simulador 4
2.1. Flujo general 4
2.2. Tecnologías utilizadas 5
3. Modelo OSI 5
4. Lógica de programación 6
4.1. Representación de las 7 capas 6
4.2. Formato de los encabezados 6
4.3. Estructuras de datos 6
4.4. Lógica de la interfaz gráfica 7
5. Diccionario de funciones 7
5.1. Función principal 7
5.2. Funciones de encapsulamiento 7
5.3. Funciones de desencapsulamiento 8
5.4. Funciones auxiliares 8
5.5. Métodos principales de la interfaz (SimuladorOSI) 9
6. Proceso de encapsulamiento (PC-A) 10
6.1. Explicación capa por capa 10
7. Proceso de desencapsulamiento (PC-B) 11
7.1. Demostración de la recuperación del mensaje 12
8. Función transmitir() 13
9. Uso de Base64 14
10. Manual de usuario 15
10.1. Requisitos para ejecutar el programa 15
10.2. Instalación y ejecución del código 15
10.3. Ingreso del mensaje 16
10.4. Inicio de la transmisión 17
10.5. Visualización del encapsulamiento 18
10.6. Visualización del desencapsulamiento 21

10.7. Recuperación del mensaje en PC-B 23
11. Análisis de resultados 23
11.1. Secuencia completa de capturas 24
11.2. Resultados obtenidos 25
11.3. Prueba con ruido en el medio 25
11.4. Pruebas unitarias y depuración 26
12. Conclusiones técnicas 27
13. Bibliografía 27
14. Evidencias de trabajo colaborativo 28
15. Declaración de uso de Inteligencia Artificial 28

1. Introducción

El Modelo OSI (Open Systems Interconnection), definido por la ISO en la norma ISO/IEC 7498-1, divide la comunicación entre computadoras en siete capas. Cada capa cumple una función concreta y agrega su propia información de control a los datos que recibe de la capa superior; este proceso se conoce como encapsulamiento. En el equipo receptor ocurre el proceso inverso, el desencapsulamiento, en el que cada capa lee, verifica y retira la información que agregó su capa par en el emisor.

Estos procesos son difíciles de observar en una red real, porque ocurren dentro del sistema operativo y en milisegundos. Por eso el grupo desarrolló un simulador interactivo en Python que representa de forma visual cómo se transforma un mensaje escrito en PC-A desde la capa de Aplicación hasta los bits de la capa Física, cómo esos bits viajan por el cable y cómo PC-B reconstruye el mensaje original subiendo desde la capa Física hasta la de Aplicación.

1.1. Objetivos

- Representar visualmente las 7 capas del Modelo OSI en un emisor (PC-A) y un receptor (PC-B).
- Mostrar, paso a paso, la información que cada capa agrega durante el encapsulamiento.
- Mostrar cómo cada capa verifica y retira su información durante el desencapsulamiento.
- Demostrar que el mensaje recuperado en PC-B es idéntico al enviado por PC-A.
- Aplicar mecanismos reales: codificación Base64, checksum de transporte y CRC32 en la trama.

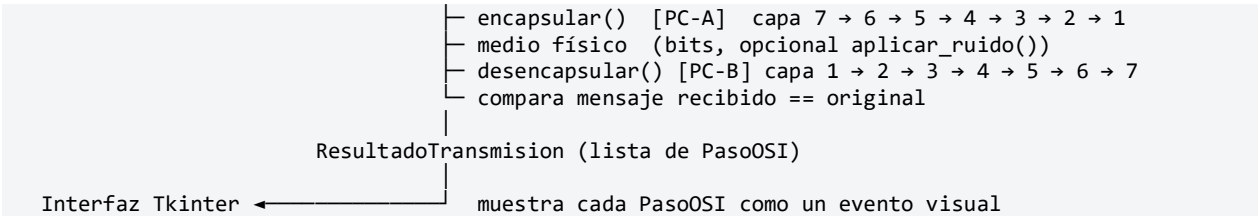
2. Arquitectura del simulador

El programa se organizó en dos módulos para separar la lógica de red de la interfaz gráfica. Así la lógica puede probarse de forma automática sin abrir ventanas y la interfaz solo se encarga de mostrar los resultados.

Archivo	Responsabilidad
osi_core.py	Lógica del Modelo OSI: definición de las 7 capas, estructuras de datos, funciones de encapsulamiento y desencapsulamiento por capa, Base64, checksum, CRC32, conversión a bits y la función principal transmitir(). También puede ejecutarse en modo consola.
simulador_osi.py	Interfaz gráfica con Tkinter: pilas de capas de PC-A y PC-B, bloques de la PDU, animación de bits en el cable, panel de detalle, bitácora y controles (Transmitir, Paso a paso, Reiniciar, Ruido, Velocidad).
tests/test_osi_core.py	15 pruebas unitarias (unittest) que verifican cada parte de la lógica.
capturas/	Capturas del simulador en funcionamiento (generadas con --capturas).
README.md / requirements.txt	Documentación del repositorio y dependencias opcionales.

2.1. Flujo general

```
Usuario escribe mensaje ──► transmitir(mensaje)
                                  │
                                  ├─ establecer_sesion()          → ID de sesión compartido
```



2.2. Tecnologías utilizadas

Herramienta	Uso en el proyecto
Python 3.10+	Lenguaje de programación del simulador.
tkinter / ttk	Ventana, botones, Canvas (dibujo de capas y animación) y cuadros de texto.
base64	Codificación de datos en la capa de Presentación.
zlib.crc32	Cálculo del FCS (Frame Check Sequence) en la capa de Enlace de Datos.
re (expresiones regulares)	Lectura de los encabezados al desencapsular.
dataclasses	Estructuras Dispositivo, PasoOSI y ResultadoTransmision.
unittest	Pruebas automáticas y depuración.
Pillow (opcional)	Captura automática de pantallas del simulador.
Git / GitHub	Control de versiones y trabajo colaborativo.

3. Modelo OSI

La tabla siguiente resume las 7 capas tal como se representan en el simulador, con su unidad de datos (PDU), su función y la información concreta que el programa agrega en cada una.

N.º	Capa	PDU	Función	Qué agrega el simulador
7	Aplicación	Datos	Interfaz con el usuario y servicios de red.	Servicio CHAT, emisor y receptor
6	Presentación	Datos	Formato, codificación, cifrado y compresión.	Texto en UTF-8 codificado en Base64
5	Sesión	Datos	Establece, mantiene y termina el diálogo.	ID de sesión y estado ACTIVA
4	Transporte	Segmento	Comunicación extremo a extremo, puertos y fiabilidad.	Protocolo TCP, puertos, secuencia, longitud y checksum
3	Red	Paquete	Direccionamiento lógico y enrutamiento.	IP origen, IP destino, TTL
2	Enlace de Datos	Trama	Direccionamiento físico y detección de errores.	MAC destino, MAC origen, tipo y FCS (CRC32)
1	Física	Bits	Transmisión de señales por el	Conversión de la trama en 0 y 1

			medio.	

Parámetro	PC-A (emisor)	PC-B (receptor)
Dirección IP	192.168.1.10	192.168.1.20
Dirección MAC	AA:AA:AA:00:00:0A	BB:BB:BB:00:00:0B
Puerto TCP	49152 (puerto efímero)	8080 (servicio de chat)

4. Lógica de programación

4.1. Representación de las 7 capas

Las capas se representan con una lista de diccionarios llamada CAPAS_OSI. Cada diccionario guarda el número, el nombre, la PDU, el color con el que se dibuja y una descripción de su función. La interfaz recorre esta lista para dibujar las dos pilas, por lo que las 7 capas se definen en un solo lugar.

```
CAPAS_OSI = [
    {"numero": 7, "nombre": "Aplicación",   "pdu": "Datos",    "color": "#E74C3C", ...},
    {"numero": 6, "nombre": "Presentación", "pdu": "Datos",    "color": "#E67E22", ...},
    {"numero": 5, "nombre": "Sesión",       "pdu": "Datos",    "color": "#F1C40F", ...},
    {"numero": 4, "nombre": "Transporte",   "pdu": "Segmento", "color": "#2ECC71", ...},
    {"numero": 3, "nombre": "Red",          "pdu": "Paquete",  "color": "#1ABC9C", ...},
    {"numero": 2, "nombre": "Enlace de Datos", "pdu": "Trama", "color": "#3498DB", ...},
    {"numero": 1, "nombre": "Física",       "pdu": "Bits",     "color": "#9B59B6", ...},
]
ORDEN_ENCAPSULAMIENTO    = [7, 6, 5, 4, 3, 2, 1]
ORDEN_DESENCAPSULAMIENTO = [1, 2, 3, 4, 5, 6, 7]
```

El orden del recorrido se controla con dos listas. Las funciones encapsular() y desencapsular() recorren esas listas con un bucle for y, en cada vuelta, llaman a la función propia de la capa a través de un diccionario {número de capa: función}. Así el código de cada capa queda aislado y el orden del modelo OSI es explícito.

4.2. Formato de los encabezados

Para que el alumno pueda leer la PDU, cada encabezado se escribe como texto con el formato [Lx|campo=valor;campo=valor]. La capa de Enlace agrega además un tráiler [FCS|crc32=...] al final de la trama, igual que Ethernet. Al desencapsular, la función extraer_encabezado() usa una expresión regular para ubicar el encabezado al inicio de la PDU, convertir sus campos en un diccionario y devolver el resto de los datos a la capa superior.

4.3. Estructuras de datos

Estructura	Tipo	Descripción
Dispositivo	dataclass	Equipo final con nombre, IP, MAC y puerto (PC_A y PC_B).
PasoOSI	dataclass	Registro de lo que pasó en una capa: capa, nombre, PDU, acción,

		dispositivo, información agregada/retirada, contenido resultante y explicación.
ResultadoTransmision	dataclass	Resultado de transmitir(): pasos, bits enviados/recibidos, mensaje recibido, éxito y error.
ErrorOSI	Exception	Error que lanza una capa cuando detecta un problema (FCS incorrecto, IP o MAC ajena, sesión desconocida). Indica en qué capa ocurrió.

4.4. Lógica de la interfaz gráfica

La clase SimuladorOSI (hereda de tk.Tk) llama a transmitir() una sola vez y convierte el resultado en una lista de eventos: inicio, 7 pasos de encapsulamiento, transmisión por el medio, 7 pasos de desencapsulamiento y fin (o error). El botón «Transmitir» recorre los eventos automáticamente con self.after() según la velocidad elegida; «Paso a paso» avanza un evento por clic. Cada evento redibuja el Canvas: la capa activa se resalta con su color, las completadas muestran ✓ y en el centro se dibuja la PDU como bloques de colores (L2 | L3 | L4 | L5 | L6 | L7 | DATOS | FCS) que crecen al encapsular y se reducen al desencapsular.

5. Diccionario de funciones

A continuación se documentan las funciones principales del código, indicando su objetivo, parámetros, resultado y la parte del proceso OSI en la que intervienen.

5.1. Función principal

Función	Objetivo	Parámetros	Resultado	Proceso OSI
`transmitir()`	Ejecuta la comunicación completa de PC-A a PC-B: establece la sesión, encapsula, transmite los bits, desencapsula y compara.	mensaje (str), origen, destino (Dispositivo), ruido (bool)	ResultadoTransmision con todos los pasos y el mensaje recibido	Todo el proceso (capas 7→1 y 1→7)
`encapsular()`	Recorre las capas 7 a 1 en el emisor llamando a la función de cada capa.	mensaje, origen, destino, id_sesion	(lista de PasoOSI, bits)	Encapsulamiento en PC-A
`desencapsular()`	Recorre las capas 1 a 7 en el receptor; detiene el proceso si una capa detecta un error.	bits, receptor, id_sesion	(lista de PasoOSI, mensaje)	Desencapsulamiento en PC-B

5.2. Funciones de encapsulamiento

Función	Objetivo	Parámetros	Resultado	Proceso OSI
`encapsular_aplicacion()`	Agrega el encabezado de aplicación (servicio, emisor, receptor).	mensaje, origen, destino	[L7|...] + mensaje	Capa 7
`encapsular_presentacio`	Convierte los datos a UTF-8 y	datos (str)	[L6|...] + Base64	Capa 6

n()	los codifica en Base64.			
encapsular_sesion()	Marca los datos con el ID de la sesión activa.	datos, id_sesion	[L5|...] + datos	Capa 5
encapsular_transporte()	Crea el segmento con puertos, secuencia, longitud y checksum.	datos, origen, destino, secuencia	Segmento [L4|...]	Capa 4
encapsular_red()	Crea el paquete con IP origen/destino y TTL.	segmento, origen, destino, ttl	Paquete [L3|...]	Capa 3
encapsular_enlace()	Crea la trama: MAC destino/origen al inicio y FCS (CRC32) al final.	paquete, origen, destino	Trama [L2|...]...[FCS|...]	Capa 2
encapsular_fisica()	Convierte la trama completa en una cadena de bits.	trama (str)	Cadena de '0' y '1'	Capa 1

Todas las funciones de encapsulamiento devuelven una tupla (pdu_nueva, texto_agregado): la primera parte se entrega a la capa inferior y la segunda se muestra al usuario como «Información agregada».

5.3. Funciones de desencapsulamiento

Función	Objetivo	Parámetros	Resultado	Proceso OSI
desencapsular_fisica()	Agrupa los bits de 8 en 8 y reconstruye la trama.	bits (str)	Trama (texto)	Capa 1
desencapsular_enlace()	Recalcula el CRC32 y lo compara con el FCS; verifica la MAC destino; retira encabezado y tráiler.	trama, receptor	Paquete o ErrorOSI(2)	Capa 2
desencapsular_red()	Verifica que la IP destino sea la del receptor y retira el encabezado.	paquete, receptor	Segmento o ErrorOSI(3)	Capa 3
desencapsular_transporte()	Valida puerto, longitud y checksum; retira el encabezado TCP.	segmento, receptor	Datos o ErrorOSI(4)	Capa 4
desencapsular_sesion()	Comprueba que el ID de sesión sea el establecido.	datos, id_sesion	Datos o ErrorOSI(5)	Capa 5
desencapsular_presentacion()	Decodifica Base64 y restaura el texto UTF-8.	datos (str)	Datos originales + [L7|...]	Capa 6
desencapsular_aplicacion()	Retira el encabezado de aplicación y entrega el mensaje.	datos (str)	Mensaje original	Capa 7

5.4. Funciones auxiliares

Función	Objetivo	Parámetros	Resultado	Proceso OSI
codificar_base64()	Convierte texto → bytes UTF-8	texto (str)	Cadena Base64	Capa 6

	→ Base64.			
`decodificar_base64()`	Operación inversa: Base64 → bytes → texto.	texto_b64 (str)	Texto original	Capa 6
`calcular_checksum()`	Suma de los bytes módulo 65536 (hex).	datos (str)	Ej.: '2D77'	Capa 4
`calcular_crc32()`	CRC32 de la trama con zlib (hex).	datos (str)	Ej.: 'C2CB26AF'	Capa 2
`texto_a_bits()`	Cada byte UTF-8 → 8 bits.	texto (str)	Cadena binaria	Capa 1
`bits_a_texto()`	Cada 8 bits → 1 byte → texto; valida los bits.	bits (str)	Texto	Capa 1
`aplicar_ruido()`	Invierte un bit para simular interferencia.	bits, posicion (opcional)	(bits alterados, posición)	Medio físico
`establecer_sesion()`	Genera el ID de sesión compartido.	—	Ej.: 'S-7113'	Capa 5
`crear_encabezado()`	Construye un encabezado [Lx|k=v;...].	etiqueta, campos (dict)	Texto del encabezado	Todas
`extraer_encabezado()`	Lee y separa un encabezado con expresión regular.	pdu, etiqueta, capa	(campos, resto)	Todas (desenc.)
`obtener_capa()`	Devuelve los datos de una capa por su número.	numero (int)	dict de la capa	Representación

5.5. Métodos principales de la interfaz (SimuladorOSI)

Método	Objetivo	Proceso OSI
`transmitir()`	Botón «Transmitir»: llama a osi.transmitir() y reproduce todos los pasos automáticamente.	Todo el recorrido
`paso_a_paso()` / `siguiente_paso()`	Avanza un evento por clic.	Todo el recorrido
`reiniciar()`	Limpia la simulación y vuelve al estado inicial.	—
`dibujar_escena()`	Dibuja las pilas de PC-A y PC-B, el cable y resalta la capa activa.	Representación de las 7 capas
`dibujar_pdu()`	Dibuja la PDU como bloques de colores (encabezados + datos + FCS).	Encapsulamiento / desencapsulamiento
`_mostrar_encapsulamiento()`	Muestra un paso de PC-A.	Capas 7→1
`_mostrar_medio()` / `_animar_bits()`	Anima los bits viajando por el cable.	Medio físico
`_mostrar_desencapsulamiento()`	Muestra un paso de PC-B.	Capas 1→7
`_mostrar_fin()` / `_mostrar_error()`	Compara el mensaje final o muestra la capa que detectó el error.	Recuperación del mensaje
`generar_capturas()`	Recorre la simulación y guarda una imagen por paso.	Evidencias

6. Proceso de encapsulamiento (PC-A)

Para ilustrar el proceso se usa el mensaje «Hola PC-B». La tabla muestra, para cada capa, la información agregada y la PDU resultante tal como la genera el simulador. Cada capa trata todo lo que recibe de la capa superior como datos y le antepone su propio encabezado.

Capa	PDU	Información agregada	PDU resultante
7 Aplicación	Datos	[L7|app=CHAT;de=PC-A;para=PC-B]	[L7|app=CHAT;de=PC-A;para=PC-B]Hola PC-B
6 Presentación	Datos	[L6|charset=UTF-8;cod=BASE64]	[L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==
5 Sesión	Datos	[L5|sesion=S-6305;estado=ACTIVA]	[L5|sesion=S-6305;estado=ACTIVA][L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==
4 Transporte	Segmento	[L4|proto=TCP;sport=49152;dport=8080;seq=1;len=117;chk=259A]	[L4|proto=TCP;sport=49152;dport=8080;seq=1;len=117;chk=259A][L5|sesion=S-6305;estado=ACTIVA][L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==
3 Red	Paquete	[L3|src=192.168.1.10;dst=192.168.1.20;ttl=64;proto=TCP]	[L3|src=192.168.1.10;dst=192.168.1.20;ttl=64;proto=TCP][L4|proto=TCP;sport=49152;dport=8080;seq=1;len=117;chk=259A][L5|sesion=S-6305;estado=ACTIVA][L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==
2 Enlace de Datos	Trama	[L2|dst=BB:BB:BB:00:00:0B;src=AA:AA:AA:00:00:0A;tipo=0x0800] ... [FCS|crc32=56B84E90]	[L2|dst=BB:BB:BB:00:00:0B;src=AA:AA:AA:00:00:0A;tipo=0x0800][L3|src=192.168.1.10;dst=192.168.1.20;ttl=64;proto=TCP][L4|proto=TCP;sport=49152;dport=8080;seq=1;len=117;chk=259A][L5|sesion=S-6305;estado=ACTIVA][L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==[FCS|crc32=56B84E90]
1 Física	Bits	2496 bits (312 bytes)	01011011 01001100 00110010 01111100 01100100 01110011 01110100 00111101 …

6.1. Explicación capa por capa

- **Capa 7 – Aplicación:** Se agrega el encabezado de Aplicación: servicio CHAT, emisor y receptor.
- **Capa 6 – Presentación:** Los datos se convierten a bytes UTF-8 y se codifican en Base64.
- **Capa 5 – Sesión:** Se agrega el ID de la sesión establecida entre PC-A y PC-B.
- **Capa 4 – Transporte:** Se forma el SEGMENTO: puertos TCP, secuencia, longitud y checksum.
- **Capa 3 – Red:** Se forma el PAQUETE: IP origen, IP destino y TTL.
- **Capa 2 – Enlace de Datos:** Se forma la TRAMA: MAC destino/origen al inicio y FCS (CRC32) al final.
- **Capa 1 – Física:** La trama se convierte en BITS que viajan por el medio físico.

Se observa que la PDU crece en cada capa: la trama final contiene todos los encabezados anidados, comenzando por el de Enlace (el más externo) y terminando en el FCS. Finalmente, la capa Física la convierte en 2496 bits (312 bytes).

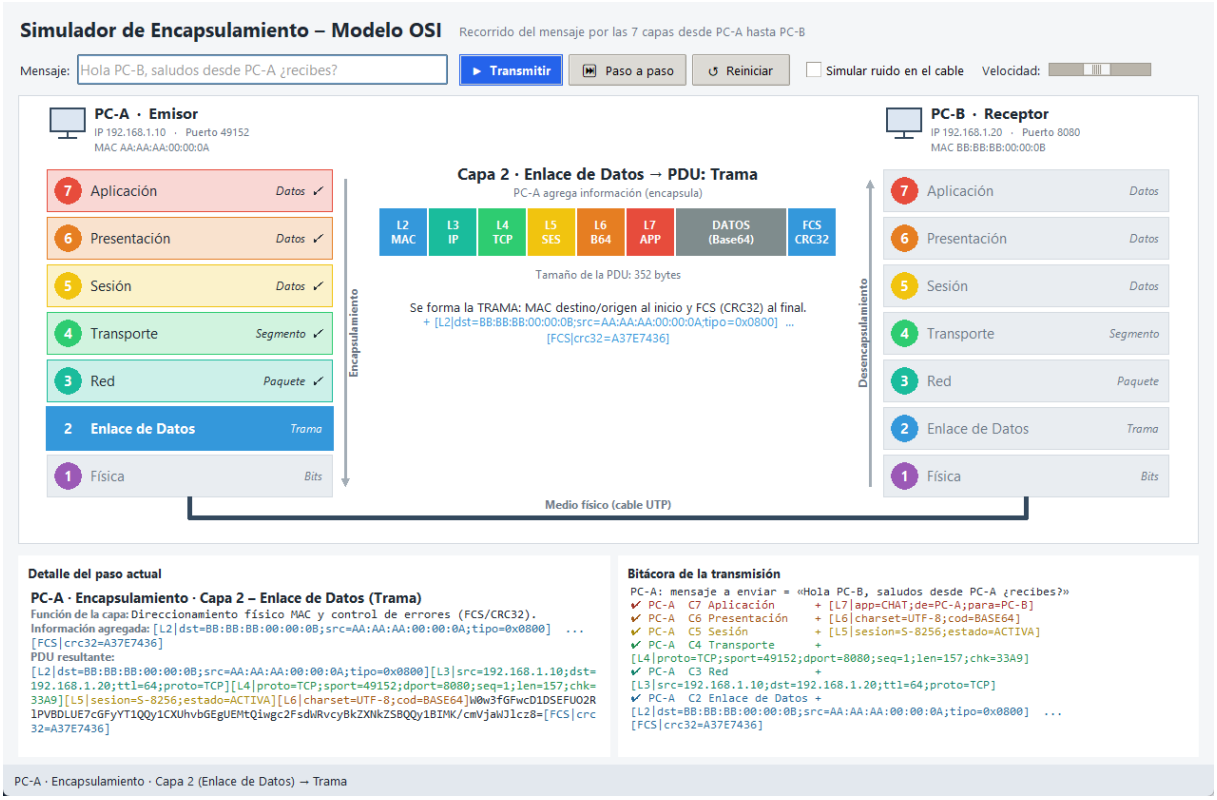


Figura 1. Encapsulamiento en la capa 2: la trama incluye todos los encabezados y el FCS.

7. Proceso de desencapsulamiento (PC-B)

PC-B recibe los bits y ejecuta el proceso inverso. Cada capa verifica la información que agregó su capa par en PC-A; si todo es correcto, la retira y entrega el resto a la capa superior.

Capa	Verificación	Información retirada	Resultado entregado
1 Física	Los bits recibidos se agrupan en bytes y se reconstruye la trama.	`2496 bits -> 312 bytes`	`[L2|dst=BB:BB:BB:00:00:0B;src=AA:AA:AA:00:00:0A;tipo=0x0800][L3|src=192.168.1.10;dst=192.168.1.20;ttl=64;proto=TCP][L4|proto=TCP;sport=49152;dport=8080;seq=1;len=117;chk=259A][L5|sesion=S-6305;estado=ACTIVA][L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==[FCS|crc32=56B84E90]`
2 Enlace de Datos	Se verifica el FCS (CRC32) y la MAC destino; se retiran encabezado y tráiler.	`[L2|dst=BB:BB:BB:00:00:0B;src=AA:AA:AA:00:00:0A;tipo=0x0800] ... [FCS|crc32=56B84E90]`	`[L3|src=192.168.1.10;dst=192.168.1.20;ttl=64;proto=TCP][L4|proto=TCP;sport=49152;dport=8080;seq=1;len=117;chk=259A][L5|sesion=S-6305;estado=ACTIVA][L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==`
3 Red	Se verifica la IP destino y se	`[L3|src=192.168.1.10;dst=192.168.1.20;ttl=64;proto=TCP]`	`[L4|proto=TCP;sport=49152;dport=8080;seq=1;len=117;chk=259A][L5|sesion=S-`

	retira el encabezado de Red.		6305;estado=ACTIVA][L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==
4 Transporte	Se valida puerto, longitud y checksum; se retira el encabezado TCP.	[L4|proto=TCP;sport=49152;dport=8080;seq=1;len=117;chk=259A]	[L5|sesion=S-6305;estado=ACTIVA][L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==
5 Sesión	Se comprueba el ID de sesión y se retira el encabezado de Sesión.	[L5|sesion=S-6305;estado=ACTIVA]	[L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==
6 Presentación	Se decodifica Base64 y se recupera el texto en UTF-8.	[L6|charset=UTF-8;cod=BASE64]	[L7|app=CHAT;de=PC-A;para=PC-B]Hola PC-B
7 Aplicación	Se retira el encabezado de Aplicación y se entrega el mensaje original.	[L7|app=CHAT;de=PC-A;para=PC-B]	Hola PC-B

7.1. Demostración de la recuperación del mensaje

Mensaje enviado por PC-A: `'Hola PC-B'`

Mensaje recibido en PC-B: `'Hola PC-B'`

Comparación en transmitir(): `resultado.exito = (mensaje_recibido == mensaje) → True`

La igualdad es exacta carácter por carácter, incluso con tildes, «ñ», signos y emojis, porque la capa de Presentación trabaja con bytes UTF-8 y Base64 es una codificación reversible sin pérdida.

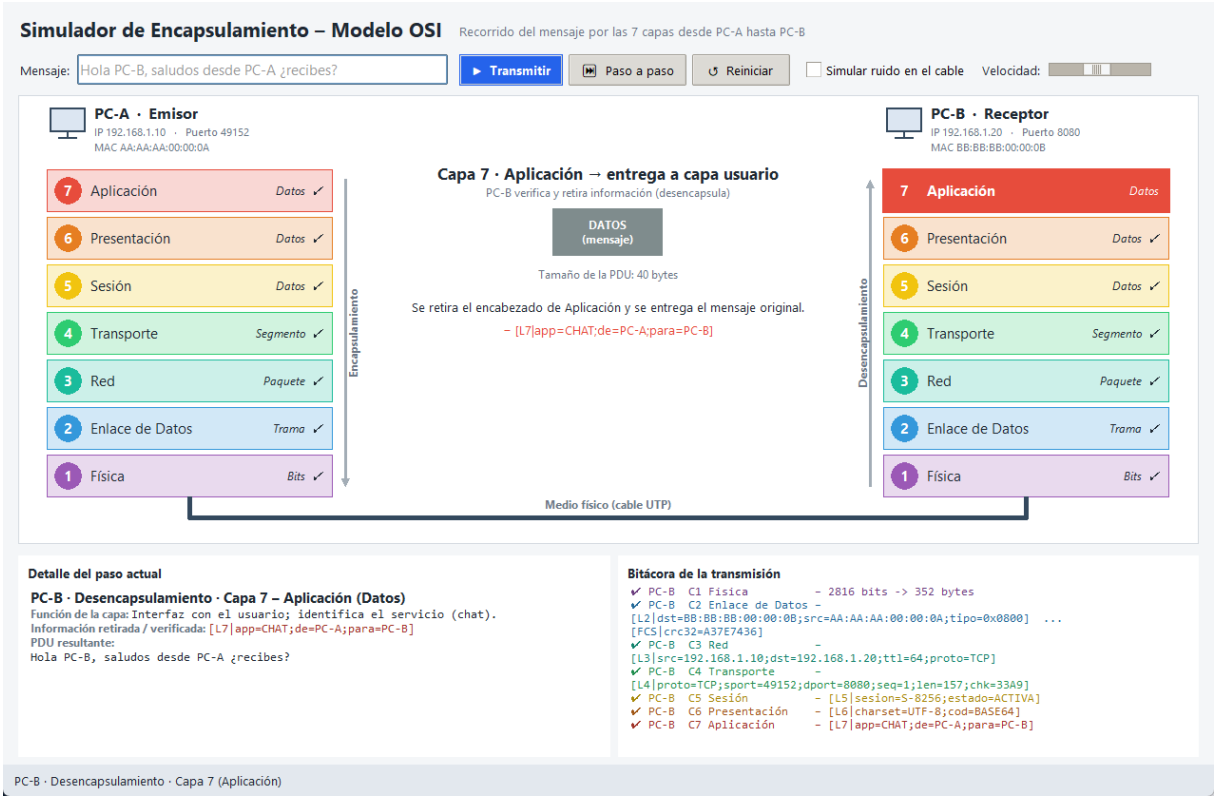


Figura 2. Desencapsulamiento en la capa 7 de PC-B: se entrega el mensaje original.

8. Función transmitir()

transmitir() es la función central del simulador. Coordina todo el proceso y devuelve un objeto ResultadoTransmision que la interfaz usa para mostrar cada paso.

```
def transmitir(mensaje, origen=PC_A, destino=PC_B, ruido=False) -> ResultadoTransmision:
    if not mensaje or not mensaje.strip():
        raise ValueError("El mensaje no puede estar vacío")

    resultado = ResultadoTransmision(mensaje_original=mensaje)
    id_sesion = establecer_sesion()                              # Capa 5: se abre la sesión

    resultado.pasos_encapsulamiento, resultado.bits_enviados = \
        encapsular(mensaje, origen, destino, id_sesion)          # PC-A: 7 → 1

    resultado.bits_recibidos = resultado.bits_enviados           # Medio físico
    if ruido:
        resultado.bits_recibidos, _ = aplicar_ruido(resultado.bits_enviados)

    try:
        resultado.pasos_desencapsulamiento, resultado.mensaje_recibido = \
            desencapsular(resultado.bits_recibidos, destino, id_sesion)   # PC-B: 1 → 7
        resultado.exito = resultado.mensaje_recibido == mensaje  # Verificación final
    except ErrorOSI as exc:
        resultado.pasos_desencapsulamiento = getattr(exc, "pasos", [])
        resultado.error, resultado.capa_error = str(exc), exc.capa
    return resultado
```

Elemento	Descripción
Parámetros	mensaje: texto a enviar. origen/destino: dispositivos (por defecto PC-A y PC-B). ruido: si es True se invierte un bit en el cable.
Resultado	ResultadoTransmision con: pasos_encapsulamiento (7), bits_enviados, bits_recibidos, pasos_desencapsulamiento (7 si no hay error), mensaje_recibido, exito, error y capa_error.
Validación	Rechaza mensajes vacíos con ValueError.
Manejo de errores	Si una capa lanza ErrorOSI, se guardan los pasos completados y la capa que falló; la interfaz la marca en rojo.
Proceso OSI	Todo el modelo: establecimiento de sesión, encapsulamiento, transmisión y desencapsulamiento.

9. Uso de Base64

Base64 es un esquema que representa datos binarios usando solo 64 caracteres imprimibles (A–Z, a–z, 0–9, «+» y «/», con «=» como relleno). Cada grupo de 3 bytes (24 bits) se divide en 4 grupos de 6 bits y cada grupo se reemplaza por un carácter de la tabla. Por eso el resultado es 4/3 del tamaño original.

En el simulador Base64 se usa en la capa de Presentación porque esta capa es la responsable de la representación y codificación de los datos. Su uso tiene tres ventajas:

- Traduce el mensaje a un formato común (UTF-8 → Base64) que ambos equipos entienden.
- Evita que caracteres del mensaje como «[», «]», «;» o «=» se confundan con los encabezados de las capas inferiores, porque Base64 no usa esos símbolos.
- Es completamente reversible: PC-B recupera exactamente los mismos bytes.

```
def codificar_base64(texto):
    return base64.b64encode(texto.encode("utf-8")).decode("ascii")

def decodificar_base64(texto_b64):
    return base64.b64decode(texto_b64.encode("ascii"), validate=True).decode("utf-8")
```

Ejemplo manual con la palabra «Hola»:

Paso	Valor
Texto	`Hola`
Bytes UTF-8 (decimal)	`72 111 108 97`
Bits (24 + 8)	`01001000 01101111 01101100 01100001`
Grupos de 6 bits	`010010 000110 111101 101100 011000 01(0000)`
Resultado Base64	`SG9sYQ==`

En la simulación, la capa 6 transforma `[L7|app=CHAT;de=PC-A;para=PC-B]Hola` PC-B en `[L6|charset=UTF-8;cod=BASE64]W0w3fGFwcD1DSEFUO2RlPVBDLUE7cGFyYT1QQy1CXUhvbGEgUEMtQg==`.

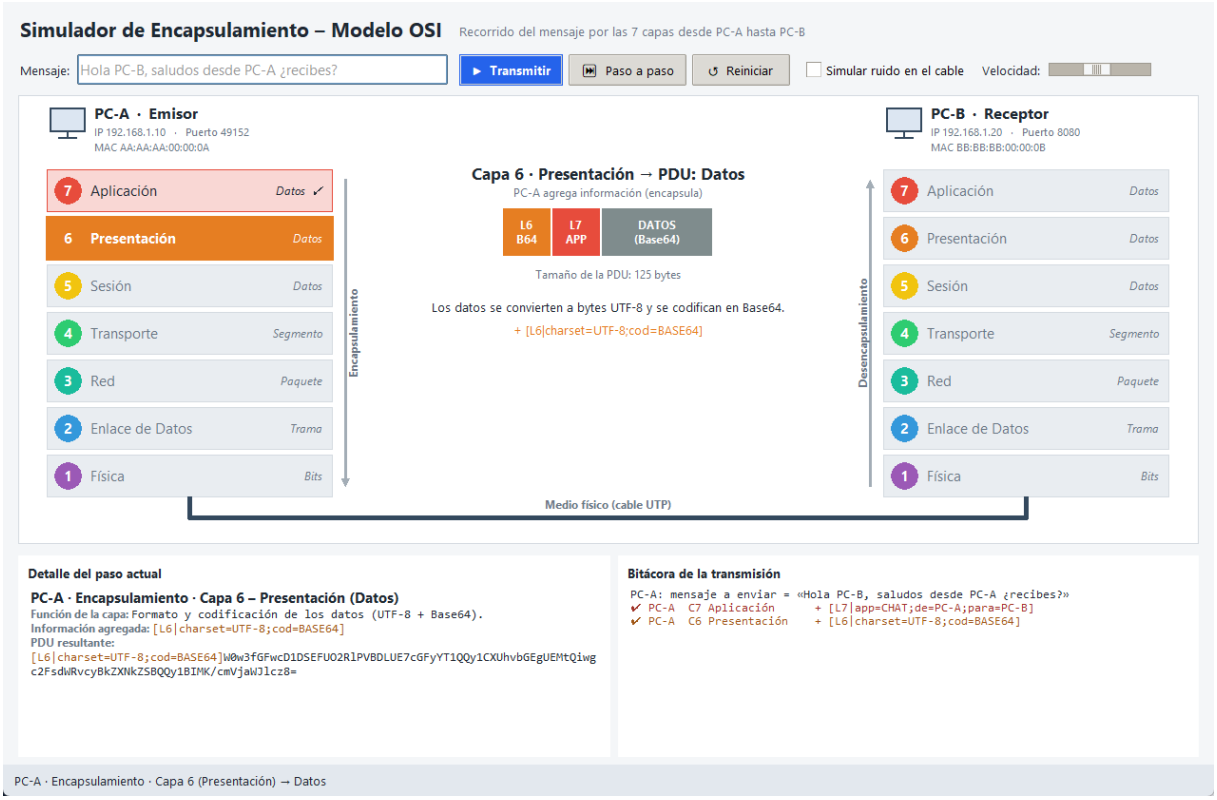


Figura 3. Capa 6: el mensaje se codifica en Base64.

10. Manual de usuario

10.1. Requisitos para ejecutar el programa

Requisito	Detalle
Sistema operativo	Windows 10/11, macOS o Linux.
Python	Versión 3.10 o superior (python.org). En Windows marcar «Add Python to PATH».
Tkinter	Incluido en Python para Windows y macOS. En Linux: sudo apt install python3-tk.
Librerías externas	Ninguna para usar el simulador. Pillow solo para generar capturas automáticas.
Pantalla	Resolución mínima recomendada 1366 × 900.

10.2. Instalación y ejecución del código

1. Descargar el proyecto desde GitHub (botón «Code → Download ZIP») o clonarlo:

```
git clone https://github.com/usuario/simulador-osi.git
cd simulador-osi
```

2. Ejecutar el simulador:

```
python simulador_osi.py
```

3. Opcional – ver el recorrido en la consola o correr las pruebas:

```
python osi_core.py "Hola PC-B"
python -m unittest -v
```

También puede abrirse la carpeta en PyCharm o VS Code y ejecutar simulador_osi.py con el botón Run.

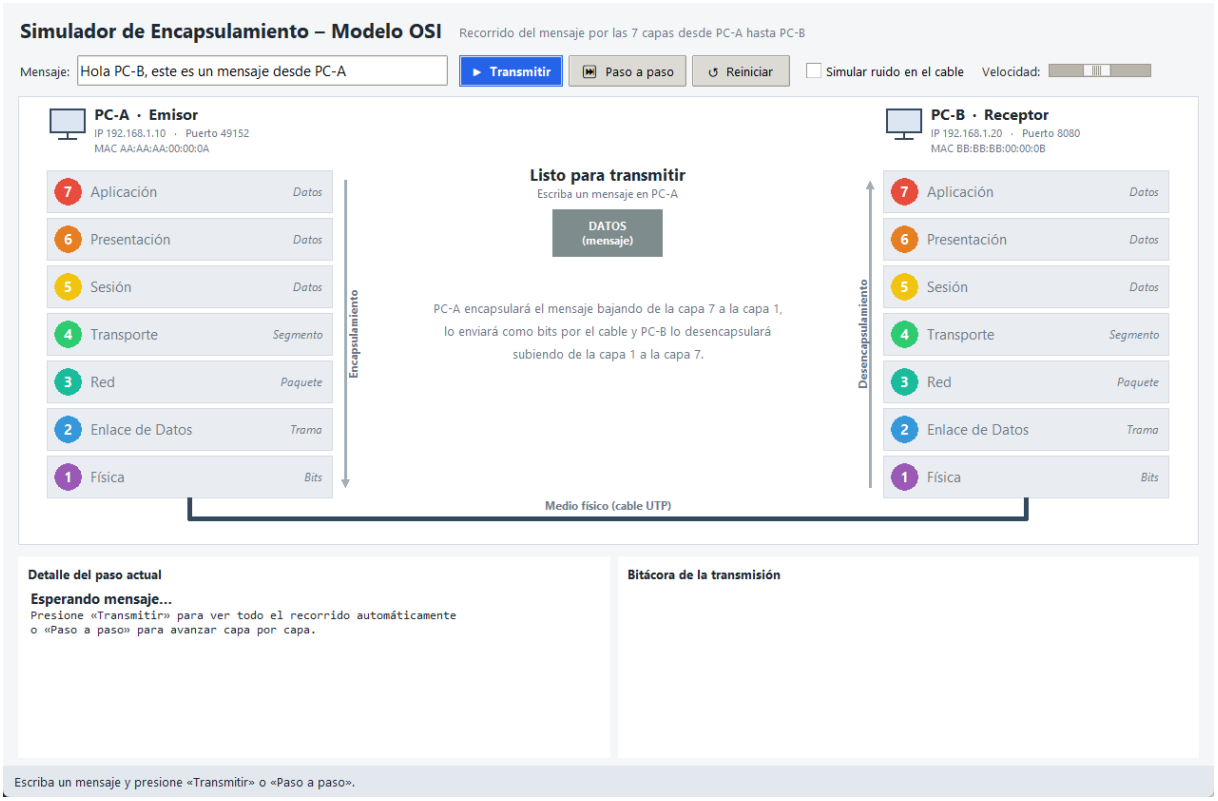


Figura 4. Pantalla inicial del simulador.

10.3. Ingreso del mensaje

En la parte superior está el campo «Mensaje». Borre el texto de ejemplo y escriba el mensaje que PC-A enviará a PC-B. Se aceptan letras, números, tildes, «ñ», signos y emojis. Si el campo está vacío el programa muestra una advertencia.

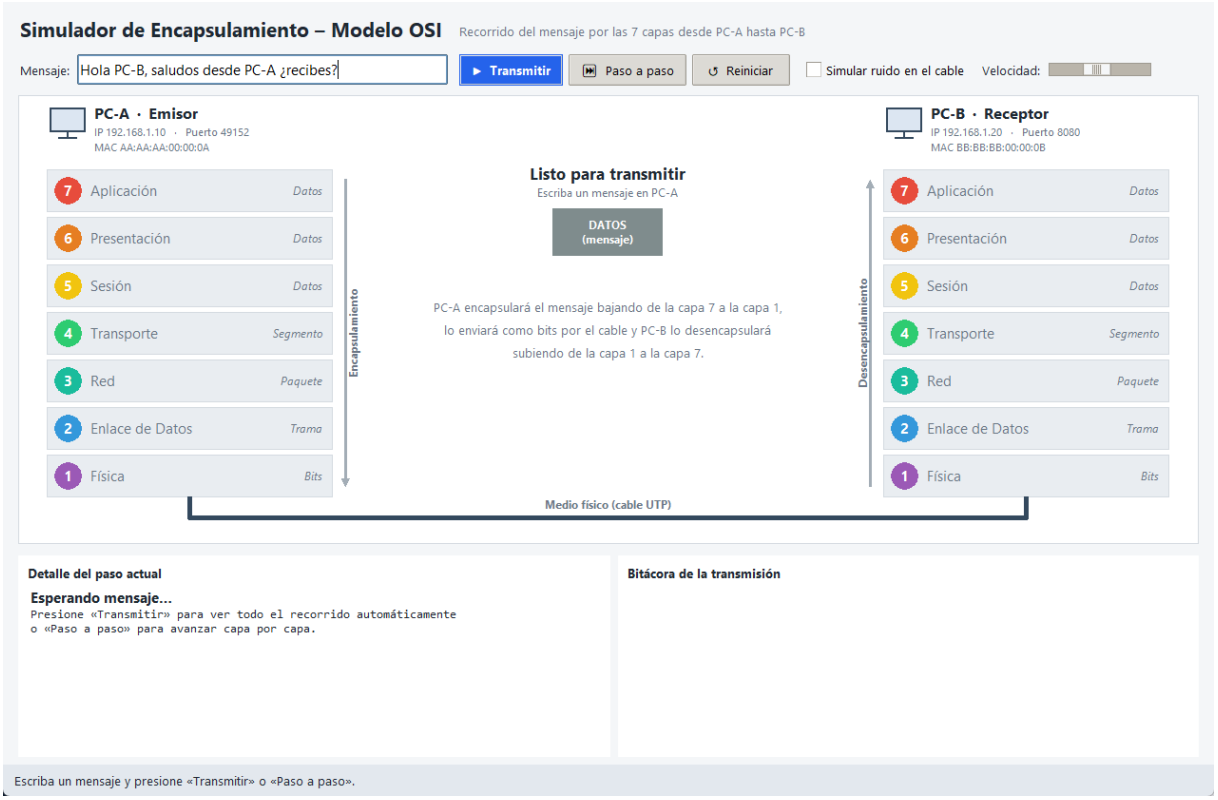


Figura 5. Mensaje escrito en el campo de entrada.

10.4. Inicio de la transmisión

Control	Función
▶ Transmitir (o tecla Enter)	Reproduce todo el recorrido de forma automática.
⏭ Paso a paso	Avanza una capa por cada clic; ideal para explicar en clase.
↺ Reiniciar	Detiene la simulación y limpia la pantalla.
Simular ruido en el cable	Invierte un bit en el medio para ver la detección de errores.
Velocidad	Control deslizante: a la derecha más rápido, a la izquierda más lento.

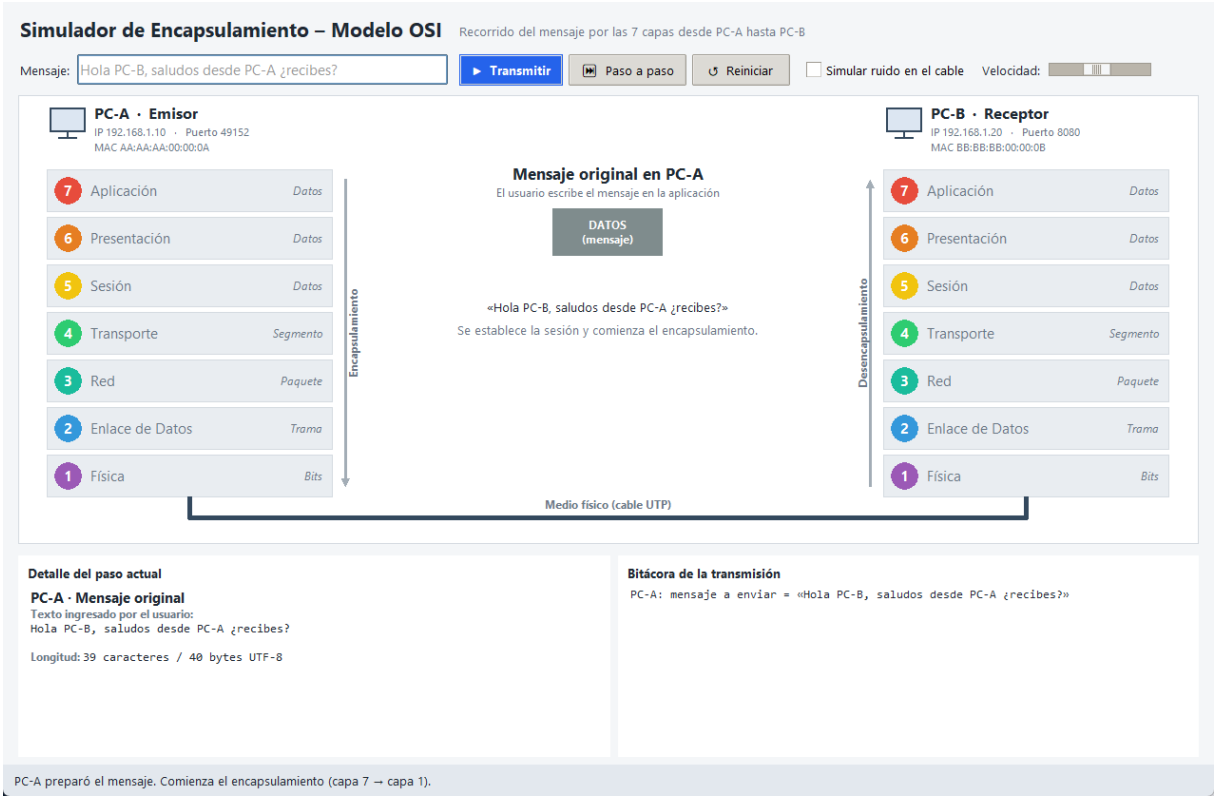


Figura 6. Inicio de la transmisión: PC-A prepara el mensaje original.

10.5. Visualización del encapsulamiento

Durante el encapsulamiento la pila de PC-A se recorre de arriba hacia abajo. La capa activa se resalta con su color, las capas terminadas muestran ✓ y en el centro la PDU se dibuja como bloques que se van agregando. El panel «Detalle del paso actual» muestra la información agregada y la PDU completa con cada encabezado en el color de su capa; la «Bitácora» conserva el historial.

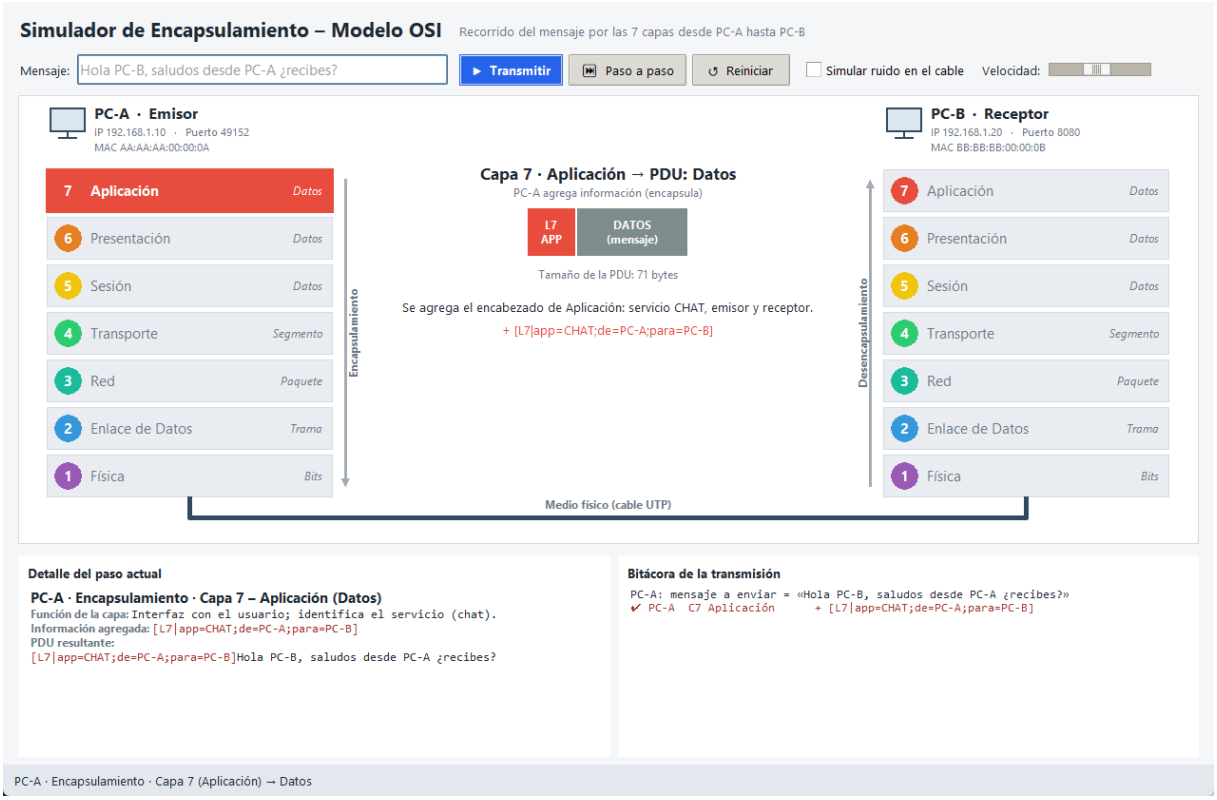


Figura 7. Capa 7 (Aplicación) en PC-A.

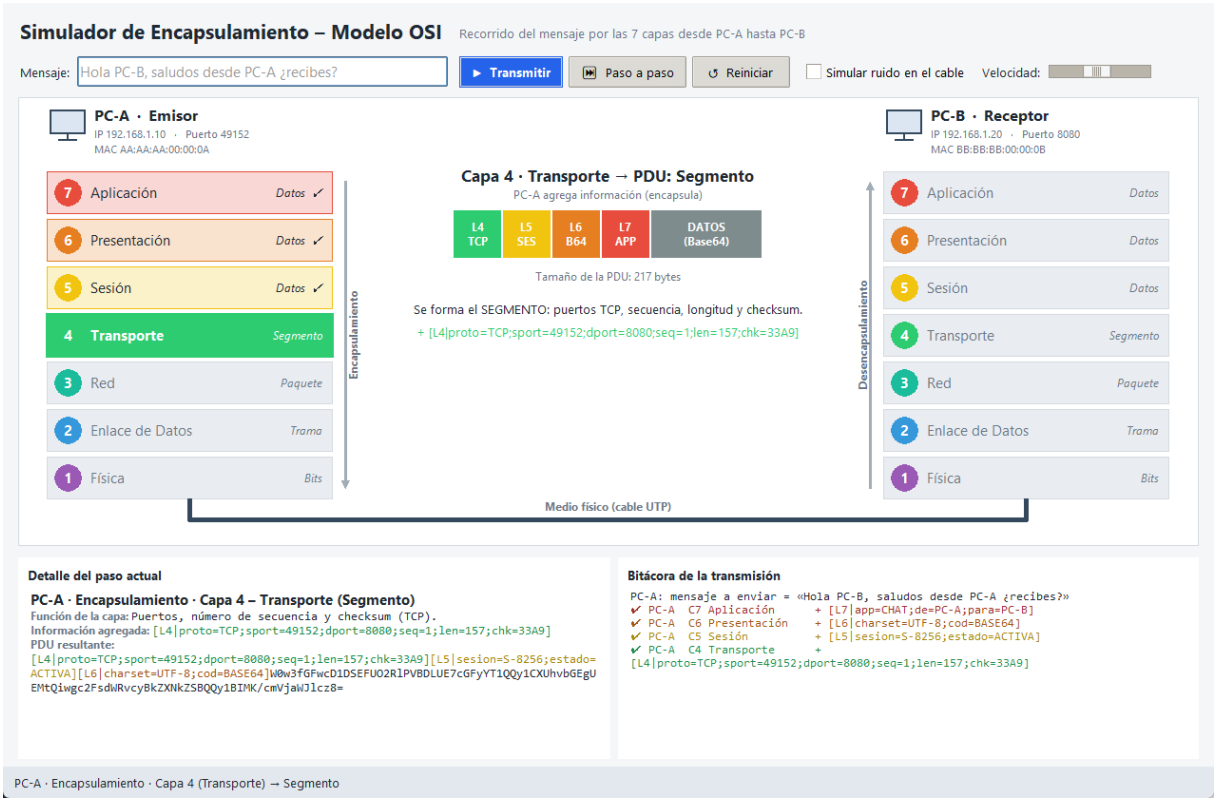


Figura 8. Capa 4 (Transporte): se forma el segmento TCP.

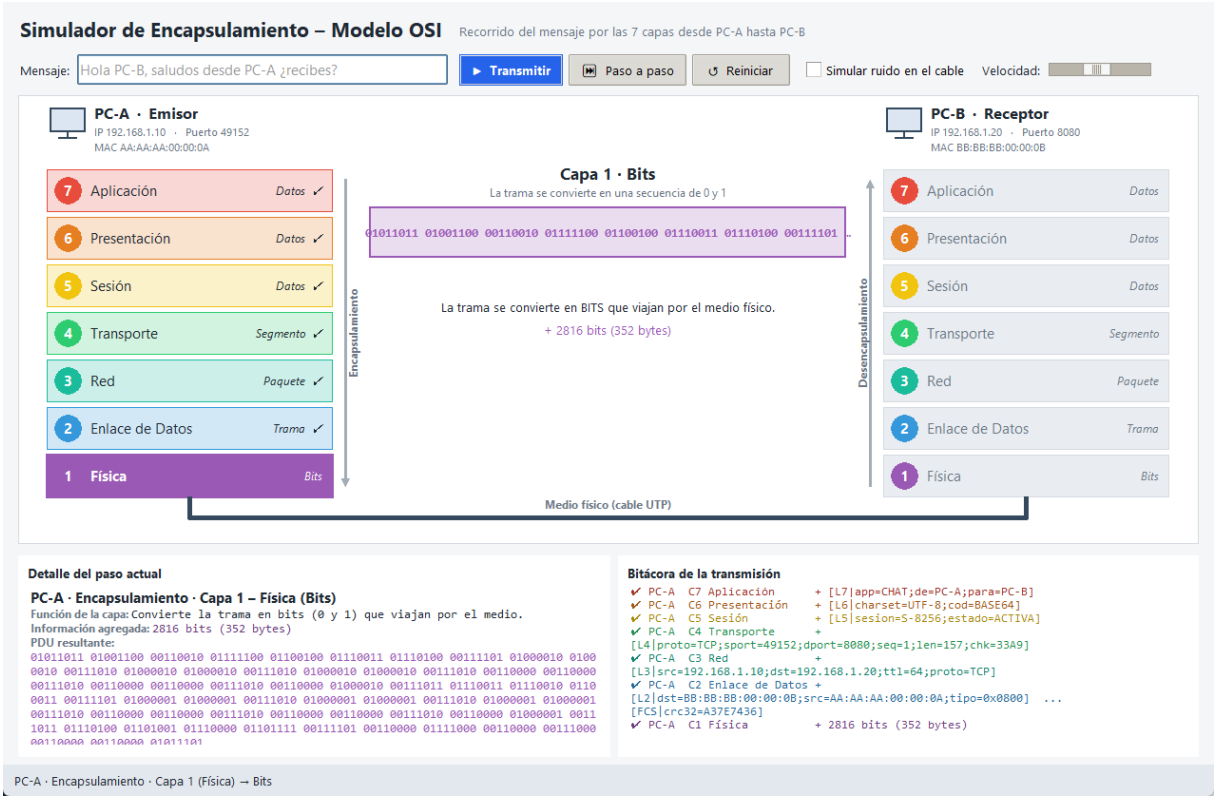


Figura 9. Capa 1 (Física): la trama se convierte en bits.

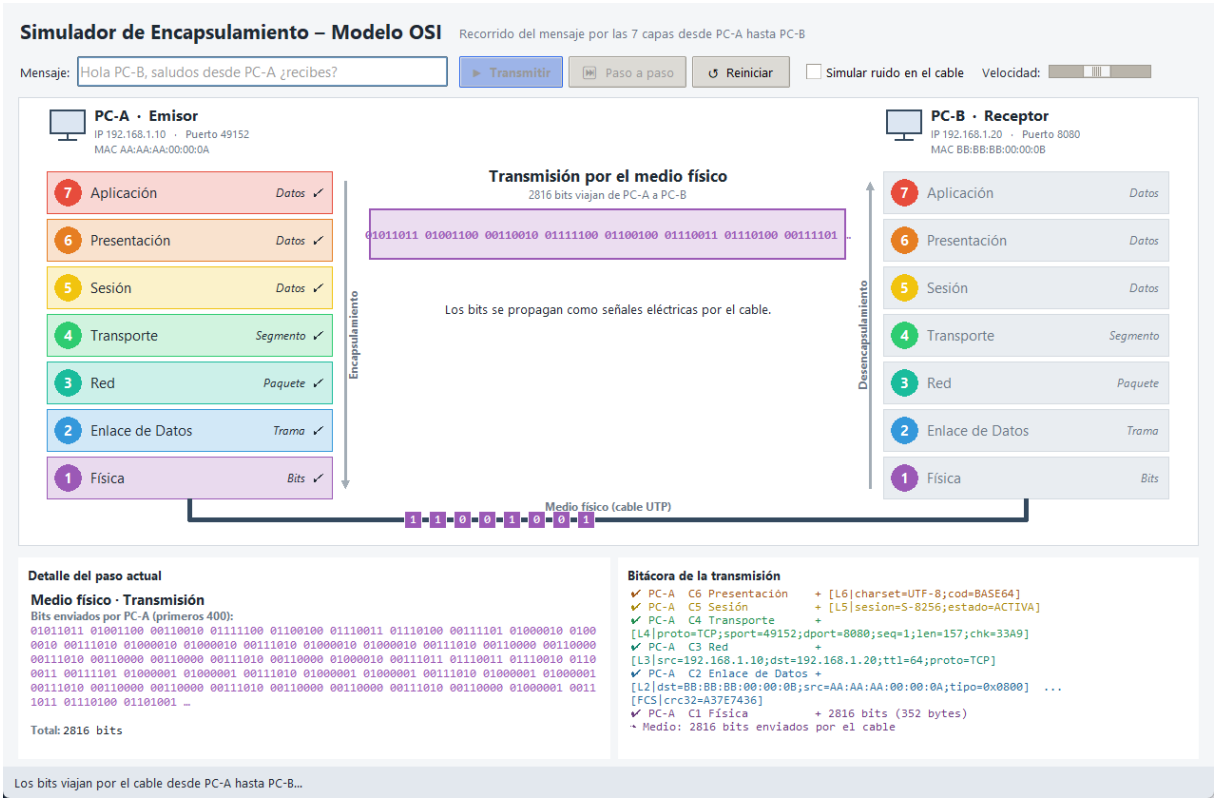


Figura 10. Los bits viajan por el cable desde PC-A hasta PC-B.

10.6. Visualización del desencapsulamiento

En PC-B la pila se recorre de abajo hacia arriba. Cada capa verifica su encabezado (FCS, MAC, IP, puerto, sesión) y lo retira; los bloques de la PDU van desapareciendo hasta quedar solo los datos.

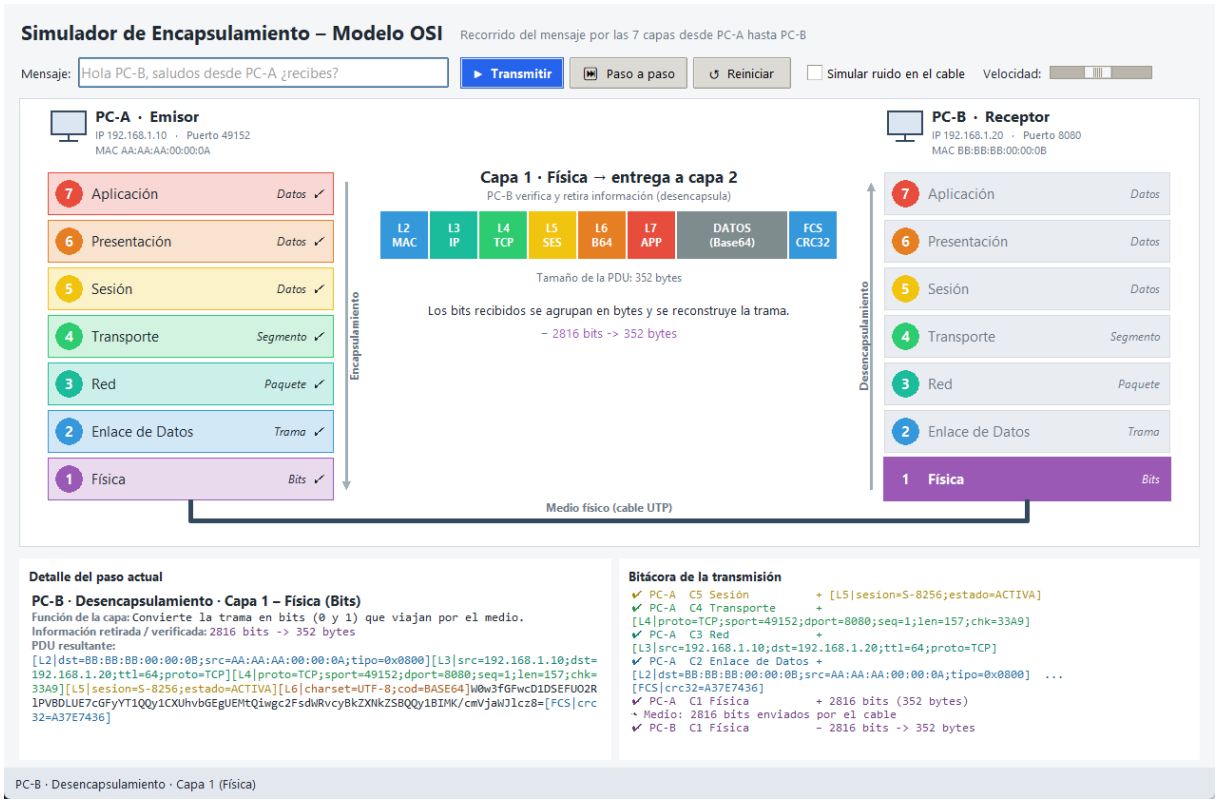


Figura 11. Capa 1 de PC-B: los bits se convierten de nuevo en la trama.

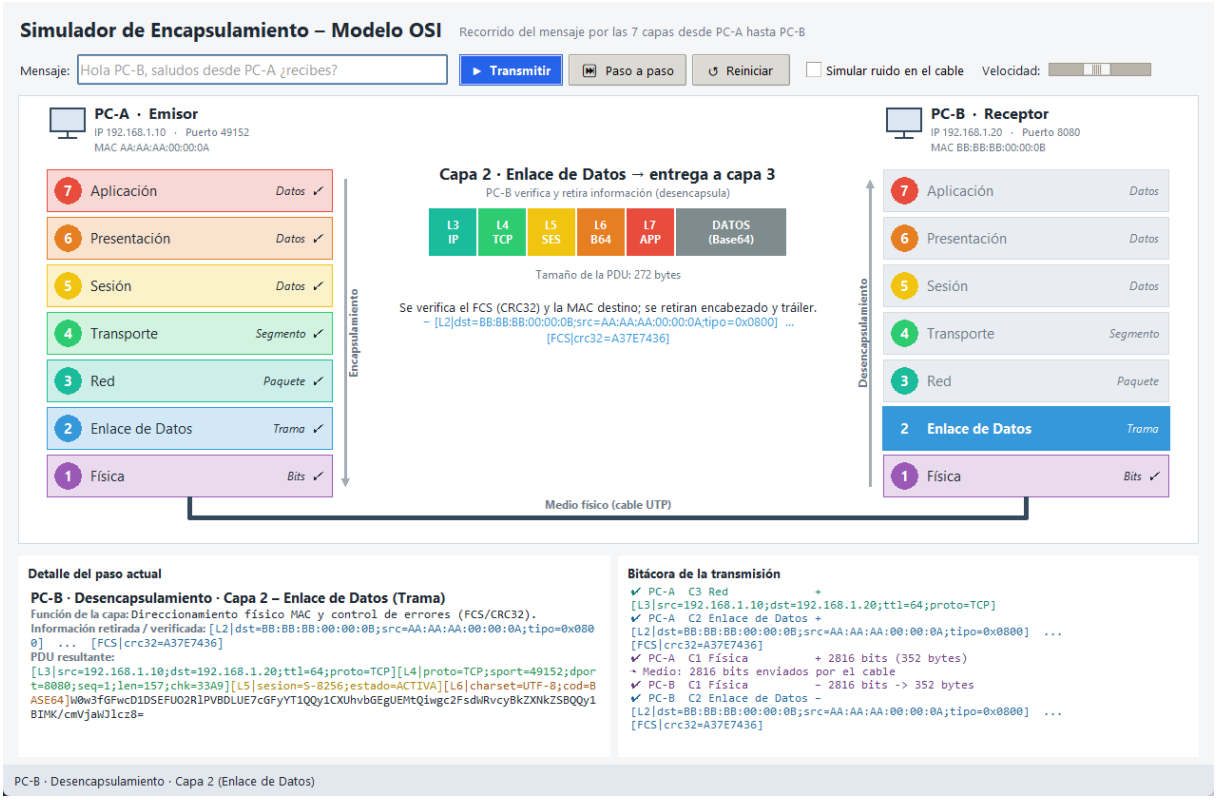


Figura 12. Capa 2 de PC-B: se verifica el CRC32 y la MAC destino.

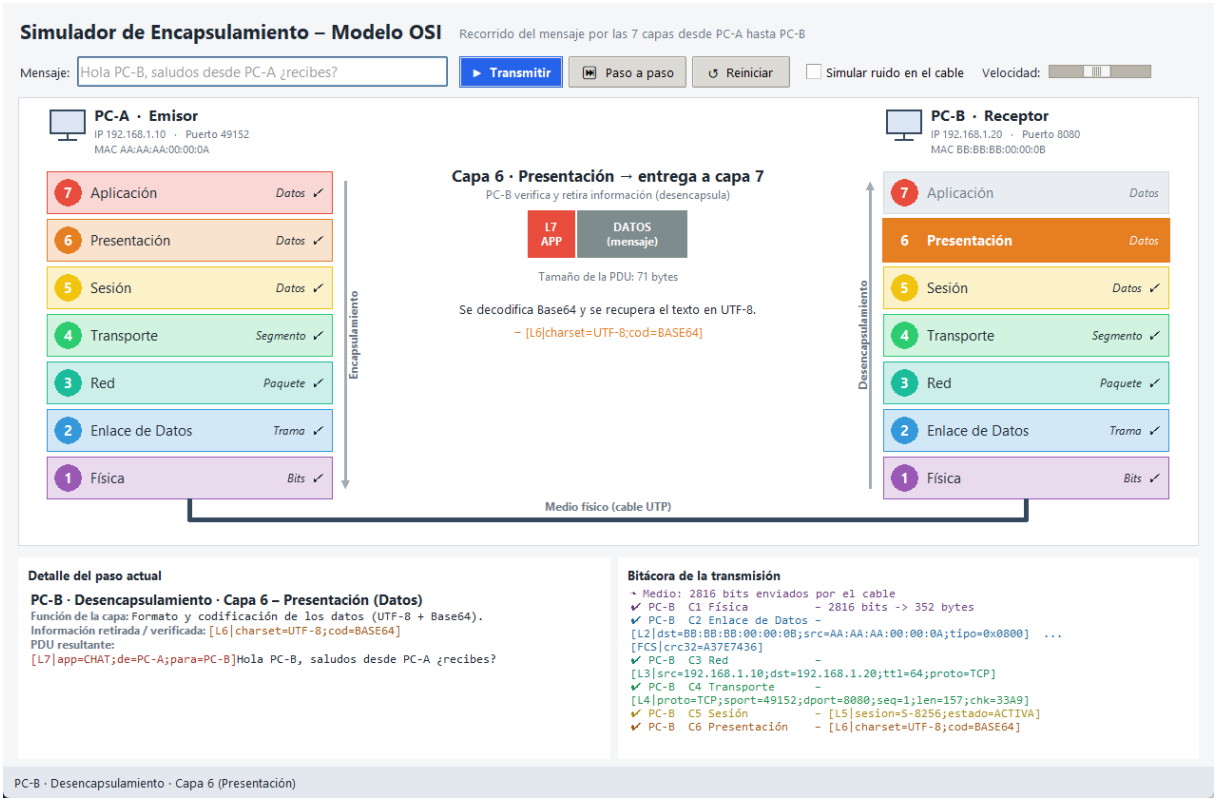


Figura 13. Capa 6 de PC-B: se decodifica Base64.

10.7. Recuperación del mensaje en PC-B

Al terminar, el simulador compara el mensaje enviado con el recibido. Si son idénticos se muestra «Mensaje recuperado en PC-B» en verde y la barra inferior confirma la transmisión exitosa. Presione «Reiniciar» o escriba un nuevo mensaje para repetir.

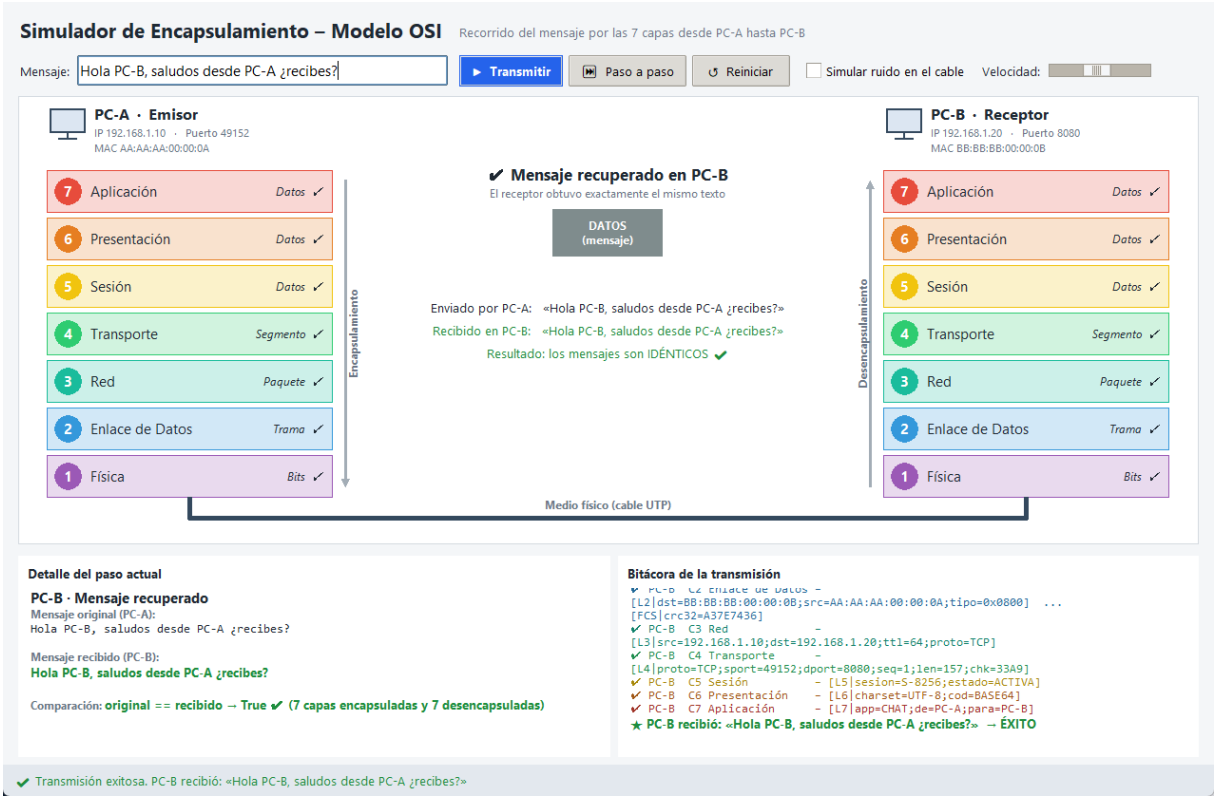


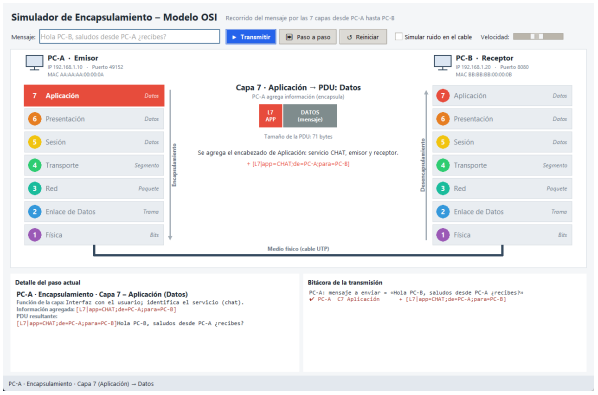
Figura 14. Mensaje recuperado correctamente en PC-B.

11. Análisis de resultados

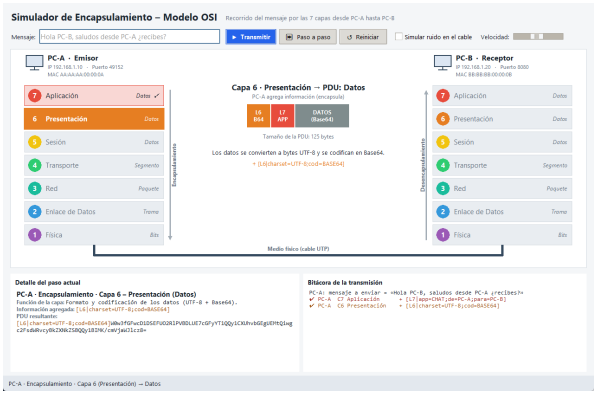
Se ejecutó el simulador con el mensaje «Hola PC-B, saludos desde PC-A ¿recibes?». La siguiente tabla resume las evidencias que demuestran cada requisito.

Requisito a demostrar	Evidencia	Resultado
El mensaje fue enviado desde PC-A	Figuras 5 y 6 (campo Mensaje y paso inicial en PC-A)	✔ Cumple
El mensaje pasó por las 7 capas	Pilas de PC-A y PC-B con ✓ en las 7 capas (Fig. 14); bitácora con 14 pasos	✔ Cumple
Se realizó el encapsulamiento	Figuras 7 a 10: la PDU crece de Datos a Bits	✔ Cumple
Se realizó el desencapsulamiento	Figuras 11 a 13: cada capa verifica y retira su encabezado	✔ Cumple
Mensaje original recuperado en PC-B	Figura 14: original == recibido → True	✔ Cumple

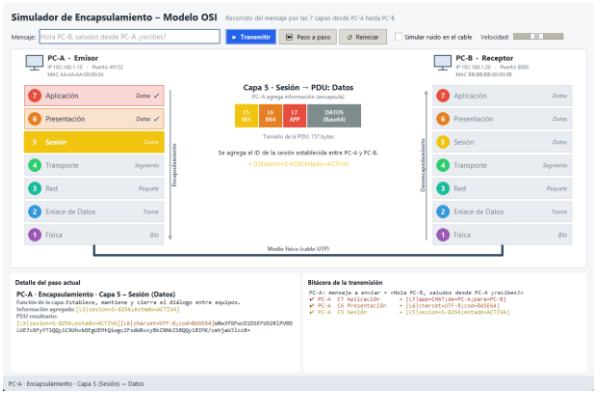
11.1. Secuencia completa de capturas



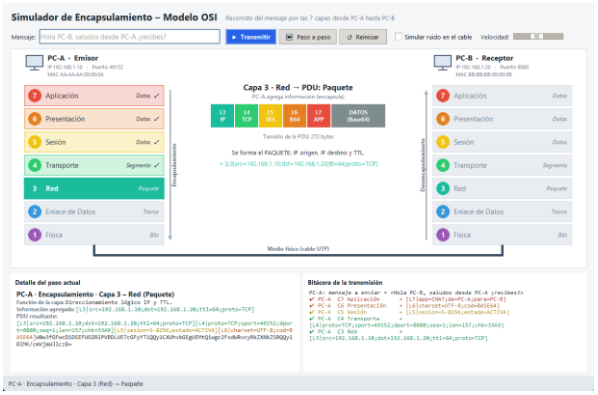
PC-A · Capa 7



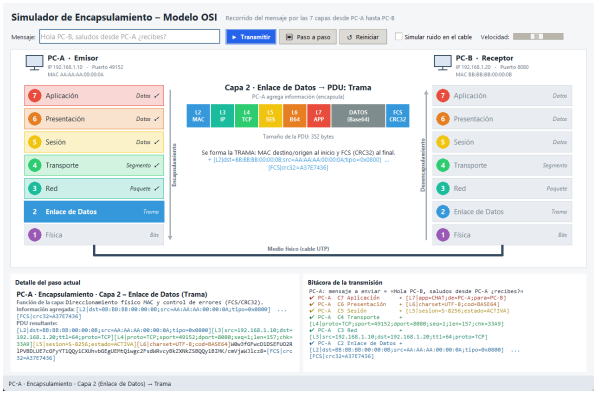
PC-A · Capa 6



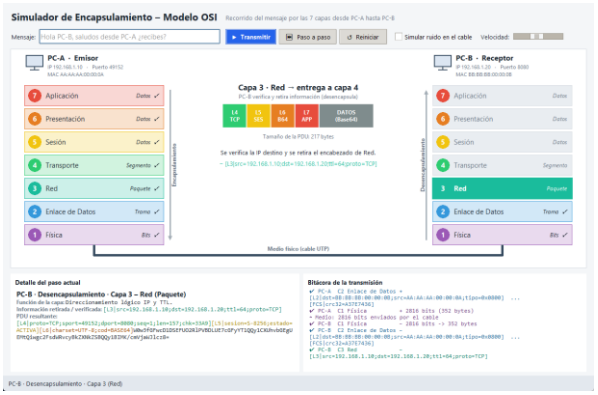
PC-A · Capa 5



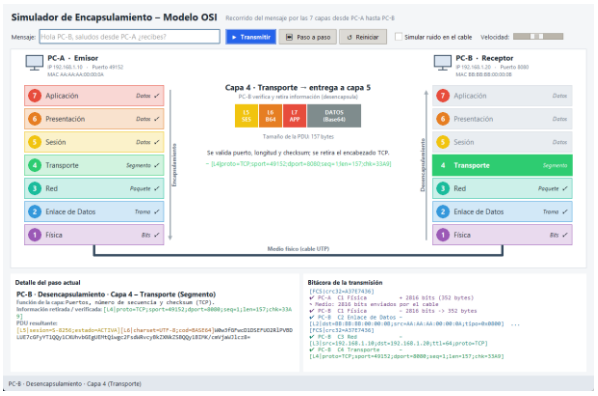
PC-A · Capa 3



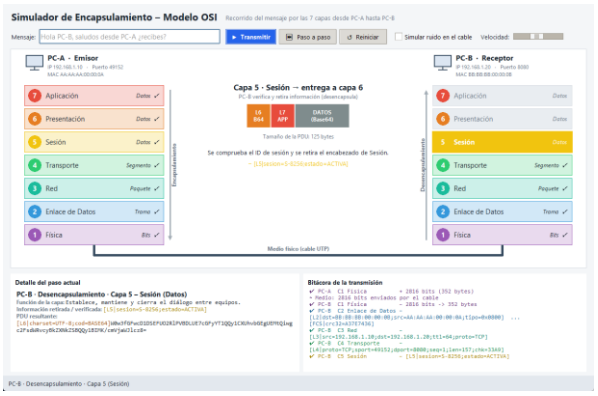
PC-A · Capa 2



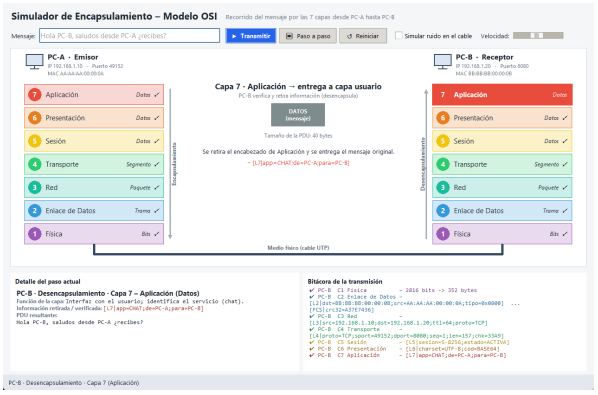
PC-B · Capa 3



PC-B · Capa 4



PC-B · Capa 5



PC-B · Capa 7

11.2. Resultados obtenidos

El mensaje de 39 caracteres (40 bytes en UTF-8, porque la «¿» ocupa 2 bytes) se transformó en una trama de 352 bytes, es decir, 2816 bits. El aumento de tamaño se debe a los encabezados de cada capa y a la codificación Base64, que agrega aproximadamente un 33 %. Esto muestra el costo (overhead) del encapsulamiento en una red real.

En PC-B cada capa validó su información: el CRC32 coincidió, la MAC y la IP destino correspondían a PC-B, el puerto 8080 estaba abierto, el checksum y la longitud del segmento eran correctos y el ID de sesión era el establecido. Tras decodificar Base64 y retirar el encabezado de Aplicación, el texto recibido fue idéntico al original.

11.3. Prueba con ruido en el medio

Como prueba adicional se activó «Simular ruido en el cable». El simulador invierte un bit durante la transmisión; la capa Física de PC-B reconstruye la trama, pero la capa de Enlace detecta que el CRC32 calculado no coincide con el FCS recibido y descarta la trama. El mensaje no llega a la aplicación, tal como ocurre en Ethernet, donde la retransmisión queda a cargo de capas superiores (TCP).

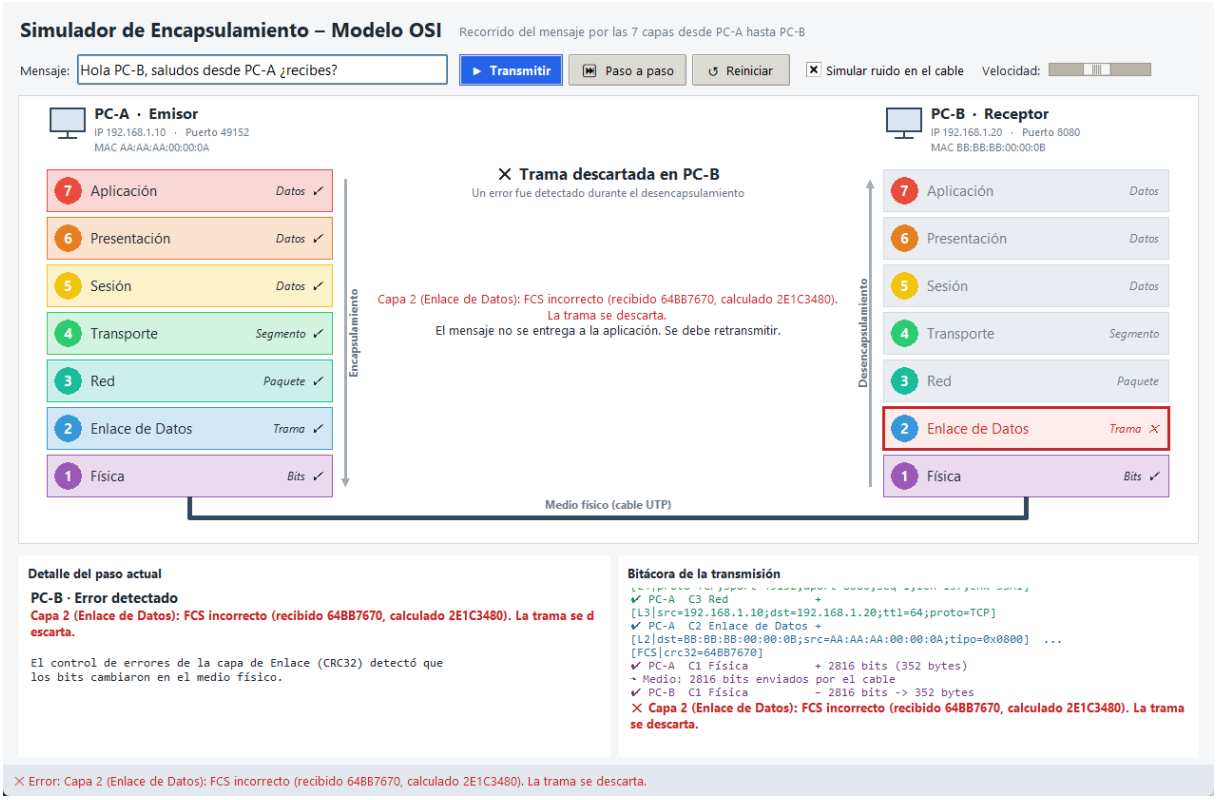


Figura 15. Prueba con ruido: la capa 2 de PC-B detecta el error y descarta la trama.

11.4. Pruebas unitarias y depuración

Para validar la lógica se escribieron 15 pruebas con el módulo unittest. Verifican la codificación Base64, la conversión a bits, el orden de las capas, la PDU de cada capa, la recuperación de mensajes con tildes, símbolos y emojis, el rechazo de mensajes vacíos y la detección de errores (ruido, MAC equivocada y sesión equivocada). Todas las pruebas se aprobaron:

```
test_ida_y_vuelta (PruebasBase64.test_ida_y_vuelta) ... ok
test_solo_caracteres_base64 (PruebasBase64.test_solo_caracteres_base64) ... ok
test_bits_ida_y_vuelta (PruebasBits.test_bits_ida_y_vuelta) ... ok
test_bits_invalidos (PruebasBits.test_bits_invalidos) ... ok
test_ocho_bits_por_byte (PruebasBits.test_ocho_bits_por_byte) ... ok
test_la_pdu_crece_hasta_enlace (PruebasEncapsulamiento.test_la_pdu_crece_hasta_enlace) ... ok
test_pdu_por_capa (PruebasEncapsulamiento.test_pdu_por_capa) ... ok
test_salida_final_son_bits (PruebasEncapsulamiento.test_salida_final_son_bits) ... ok
test_siete_capas_en_orden (PruebasEncapsulamiento.test_siete_capas_en_orden) ... ok
test_trama_tiene_encabezado_y_trailer (PruebasEncapsulamiento.test_trama_tiene_encabezado_y_trailer) ... ok
test_mac_equivocada (PruebasTransmision.test_mac_equivocada) ... ok
test_mensaje_vacio (PruebasTransmision.test_mensaje_vacio) ... ok
test_mensajes_recuperados (PruebasTransmision.test_mensajes_recuperados) ... ok
test_ruido_detectado_en_enlace (PruebasTransmision.test_ruido_detectado_en_enlace) ... ok
test_sesion_equivocada (PruebasTransmision.test_sesion_equivocada) ... ok

----------------------------------------------------------------------
Ran 15 tests in 0.007s

OK
```

12. Conclusiones técnicas

La lógica del simulador se basó en tratar cada capa del Modelo OSI como una función independiente que recibe la PDU de la capa superior y devuelve una PDU nueva. Esta decisión hizo que el código reflejara directamente el modelo teórico: el encapsulamiento es un bucle que recorre las capas 7 a 1 y el desencapsulamiento es el mismo bucle en sentido contrario. Separar la lógica (osi_core.py) de la interfaz (simulador_osi.py) permitió probar el comportamiento de red sin depender de la parte gráfica.

El simulador demostró que el encapsulamiento consiste en agregar información de control sin modificar los datos recibidos: cada capa antepone su encabezado y la capa de Enlace agrega además un tráiler. En el receptor, cada capa solo interpreta el encabezado de su capa par, lo verifica y lo retira, de modo que el mensaje llega intacto a la aplicación. La comparación final (original == recibido) confirmó que el proceso es reversible y que el mensaje se recupera sin pérdida.

La aplicación del Modelo OSI en el programa no se limitó a nombres: se usaron mecanismos reales como Base64 en Presentación, identificadores de sesión, puertos y checksum en Transporte, direcciones IP y TTL en Red, direcciones MAC y CRC32 en Enlace y la conversión a bits en la capa Física. La prueba con ruido evidenció por qué existe el control de errores: un solo bit alterado basta para que la capa 2 descarte la trama.

Python resultó adecuado para representar procesos de redes gracias a su biblioteca estándar: base64, zlib, re, dataclasses y tkinter cubrieron todas las necesidades sin instalar paquetes externos, lo que facilita ejecutar el simulador en cualquier computadora del laboratorio.

Entre las dificultades encontradas estuvieron separar correctamente los encabezados cuando el mensaje contenía caracteres como «[» o «;» (resuelto al aplicar Base64 antes de las capas inferiores), manejar caracteres con tilde y emojis que ocupan varios bytes en UTF-8, sincronizar la animación de los bits con el avance automático y mostrar la trama completa sin saturar la pantalla.

Finalmente, las pruebas y la depuración fueron fundamentales. Las pruebas unitarias detectaron errores en el orden de las capas y en la reconstrucción de bits antes de conectarlos con la interfaz, y el modo consola permitió seguir la PDU paso a paso para localizar fallos. Probar casos límite, como mensajes vacíos, texto con símbolos o una trama con ruido, dio confianza en que el simulador representa fielmente el comportamiento esperado de una red.

13. Bibliografía

Cisco Networking Academy. (2020). Introducción a redes (CCNA v7). Cisco Press.

Forouzan, B. A. (2013). Data communications and networking (5.ª ed.). McGraw-Hill.

International Organization for Standardization. (1994). ISO/IEC 7498-1:1994 Information technology — Open Systems Interconnection — Basic Reference Model. ISO.

Josefsson, S. (2006). RFC 4648: The Base16, Base32, and Base64 data encodings. IETF. https://www.rfc-editor.org/rfc/rfc4648

Kurose, J. F., & Ross, K. W. (2021). Computer networking: A top-down approach (8.ª ed.). Pearson.

Python Software Foundation. (2026). base64 — Base16, Base32, Base64, Base85 Data Encodings. https://docs.python.org/3/library/base64.html

Python Software Foundation. (2026). tkinter — Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html

Python Software Foundation. (2026). zlib — Compression compatible with gzip. https://docs.python.org/3/library/zlib.html

Tanenbaum, A. S., Feamster, N., & Wetherall, D. (2021). Computer networks (6.ª ed.). Pearson.

14. Evidencias de trabajo colaborativo

[Completar por el grupo] Registrar la participación de cada integrante y adjuntar capturas del repositorio de GitHub (historial de commits, ramas, issues o pull requests) y de las reuniones de trabajo.

Integrante	Rol / responsabilidades	Aportes (commits, secciones)	Fecha
[Integrante 1]	[Ej.: lógica de capas 7–5 y Base64]		
[Integrante 2]	[Ej.: capas 4–1, CRC32 y bits]		
[Integrante 3]	[Ej.: interfaz Tkinter y animación]		
[Integrante 4]	[Ej.: pruebas, capturas y documentación]		

Enlace del repositorio: [https://github.com/usuario/simulador-osi]

[Insertar aquí captura del historial de commits de GitHub]

[Insertar aquí capturas de reuniones / chat de coordinación del grupo]

15. Declaración de uso de Inteligencia Artificial

Declaramos que en la elaboración de este trabajo se utilizó una herramienta de inteligencia artificial generativa (Claude, de Anthropic, mediante Claude Code) como apoyo para: [marcar/ajustar lo que corresponda] generar una primera versión del código del simulador y de las pruebas unitarias, proponer la estructura del manual técnico y redactar borradores de las explicaciones.

El grupo revisó, ejecutó y probó el código, verificó que el funcionamiento corresponda a los conceptos del Modelo OSI vistos en clase, tomó las capturas de pantalla y ajustó los textos. Los integrantes asumen la responsabilidad total del contenido entregado y pueden explicar cada parte del programa.